

Lecture review

Encapsulation is one of the fundamental concepts in object-oriented programming (OOP). It describes the idea of bundling data and methods that work on that data within one unit, e.g., a class in C++. This concept is also often used to hide the internal representation, or state, of an object from the outside. This is called information hiding.

Encapsulation can be achieved by making *instance variables* of the class **private** and the access to the members is then controlled by **public member functions** of the class.

For instance, the `Student` class below has three private instance variables `erpID`, `name`, and `cgpa`.

```
class Student {  
private:  
    int erpID;  
    string name;  
    float cgpa;  
public:  
    Student(int id, string n, float c) { // constructor  
        erpID = id;  
        name = n;  
        cgpa = c;  
    }  
    void setCGPA(float c) { // setter  
        cgpa = c;  
    }  
    int getErpID() { // getter  
        return erpID;  
    }  
    string getName() { // getter  
        return name;  
    }  
    float getCGPA() { // getter  
        return cgpa;  
    }  
};
```

The public interface of the class includes a constructor that takes three parameters and initializes the instance variables `erpID`, `name`, and `cgpa`.

There is a public member functions `setCGPA()` to set the value of the instance variable `cgpa`. Such member functions are also called *setters* or *mutators*. Note that we have not provided setter functions for `erpID` and `name`. This is because we do not want to allow the user to change these values once they are initialized in the constructor.

It also has *getter* or *accessor* functions `getErpID()`, `getName()`, and `getCGPA()` to access the values of the instance variables.

Lecture review

A **class** in C++ can be used to implement an *abstract data type* (ADT). As discussed in lecture, the first step in designing an ADT is to decide upon the *API* i.e., the set of operations that can be performed on the data. The next step is to decide upon the *representation* of the data. The *instance variables* (also called *attributes*) of the class are used to represent the data. The representation of the data is hidden from the user of the ADT. The user can only access the data through the operations defined in the API. This is called *data encapsulation* or *information hiding*. The attributes and methods of a class are collectively called the *members* of the class.

A *constructor* is a special method that is used to initialize objects. It is called when an object of a class is created. It can be used to set initial values for object attributes. It can also be used to perform any other initialization tasks required when an object is created. A constructor is defined like a method, except that it has the same name as the class and has no return type. For example, the following constructors can be defined for a class implementing *rational* numbers.

- A *default constructor* that initializes the numerator to 0 and the denominator to 1.
- A constructor that takes a single integer argument and initializes the numerator to that value and the denominator to 1.
- A constructor that takes two integer arguments and initializes the numerator and denominator to those values.
- A *copy constructor* that takes a *rational* object as an argument and initializes the numerator and denominator to the values of the corresponding attributes of the argument object.

The following code shows the definitions of the four overloaded constructors described above.

```
class Rational {
    private:
        int num, den;

    public:
        // default constructor
        Rational() : num {0}, den {1} { /* empty */ }

        Rational(int n) : num {n}, den {1} { /* empty */ }

        Rational(int n, int d) : num {n}, den {d} { /* empty */ }

        // copy constructor
        Rational(const Rational& r) : num {r.num}, den {r.den} { /* empty */ }
};
```

Lecture review

Stack and Queue

Stack and *queue* are two important ADTs. A stack stores items in a last-in, first-out (LIFO) manner, whereas a queue stores items in a first-in, first-out (FIFO) manner. Following table summarizes the operations of a stack and a queue.

Stack	Queue	Description
<code>push(x)</code>	<code>enqueue(x)</code>	Insert an item x
<code>pop()</code>	<code>dequeue()</code>	Remove and return an item (most recently inserted item for stack and least recently inserted one for queue)
<code>empty()</code>	<code>empty()</code>	Return true if the stack/queue is empty
<code>size()</code>	<code>size()</code>	Return the number of items in the stack/queue

We have seen two implementations of stacks and queues in the lecture: **RAStack** and **RAQueue** using *resizable arrays*, and similarly **LLStack** and **LLQueue** using *linked lists*. Note that from the perspective of the user, the two implementations of stack (respectively queue) are indistinguishable as both provide the identical public interface. The following files are provided for your reference:

- `ra_stack.cpp` and `ll_stack.cpp` implement a stack using resizable array and linked list respectively.
- `ra_queue.cpp` and `ll_queue.cpp` implement a queue using resizable array and linked list respectively.

Copy constructor: Shallow vs Deep copy

A *copy constructor* is a special constructor that initializes a new object from an existing object. It is used to create a new object as a copy of an existing object. When copying objects that contain pointers, (as the above implementations of stack and queue do), we need to be careful about how we want to make a copy. For instance, consider the class **RAStack** that contains a pointer **data** to an array of integers. The copy constructor of **RAStack** can be implemented in following two ways:

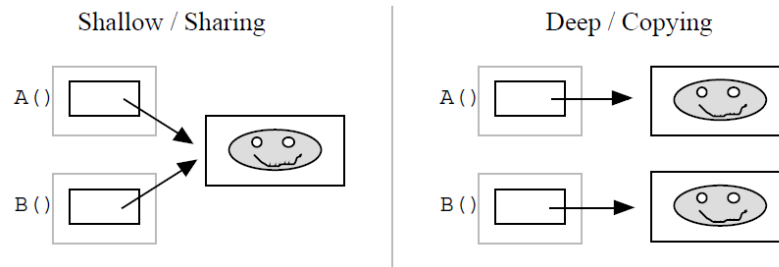
```
RAStack(const RASTack& rhs)
: arr(rhs.arr),
  sz(rhs.sz),
  cap(rhs.cap) {
}
```

```
RAStack(const RASTack& rhs)
: arr(new string[rhs.cap]),
  sz(rhs.sz),
  cap(rhs.cap) {
  for (int i = 0; i < sz; ++i)
    arr[i] = rhs.arr[i];
}
```

On the left is the *shallow copy* implementation of the copy constructor, and on the right is the *deep copy* implementation. By shallow copy, we mean that the copy constructor copies the pointer but not the object that the pointer points to. Whereas deep copy mean that the copy constructor allocate a new array and copies the elements of the array pointed to by `rhs.arr` to the new array.

Note that with shallow copy, the two objects ***this** and **rhs** will share the same array. So if we modify the array of one object, the other object will also be affected. This is not the case with deep copy as both stack objects will have their own copy of the array. So naturally, deep copy is more expensive than shallow copy as it involves allocating a new array and copying the elements of the array. But it is also safer as the two objects will not share the same array.

Following figure illustrates the difference between shallow and deep copy:



Assignment operator

The assignment operator is used to assign a new value to an existing variable. The operator can be overloaded to provide a specific implementation of the assignment for a class. For instance, consider the following implementation of the assignment operator for **RASStack**:

```
RASStack& operator=(const RASStack& rhs) {
    if (this != &rhs) {
        delete[] arr;
        arr = new string[rhs.cap];
        sz = rhs.sz;
        cap = rhs.cap;
        for (int i = 0; i < sz; ++i)
            arr[i] = rhs.arr[i];
    }
    return *this;
}
```

The concept of shallow and deep copy also applies to the assignment operator. The above implementation of the assignment operator is a deep copy implementation.

Lecture review

Public Inheritance

Public inheritance is a fundamental concept in object-oriented programming (OOP) where a class (derived/-subclass) inherits attributes and behaviors (methods) from another class (base/superclass).

```
class DerivedClass : public BaseClass {  
    // Derived class members  
};
```

The access levels of the base class members are preserved in the derived class.

- Public members of the base class remain public in the derived class.
- Protected members of the base class remain protected in the derived class.
- Private members of the base class are not accessible in the derived class.

Is-a relationship

The public inheritance establishes an “is-a” relationship between classes where one class is a specialized version of another.

If class **Dog** inherits publicly from class **Animal**, it implies that every object of class **Dog** is also an object of class **Animal**, but not vice versa.

```
class Animal {  
    // Common attributes and methods  
};  
class Dog : public Animal {  
    // Dog-specific attributes and methods  
};
```

Overriding and Virtual Functions

Overriding allows a derived class to provide a specific implementation of a method that is already defined in its base class.

Virtual functions are functions in base classes that are declared using the virtual keyword and can be overridden by derived classes. They enable runtime polymorphism, allowing the correct method implementation to be invoked dynamically based on the object type.

```
class BaseClass {  
public:  
    virtual void foo() { // virtual keyword indicates that this function can be  
        overridden  
        // Base implementation  
    }
```

```
};
class DerivedClass : public BaseClass {
public:
    void foo() override { // Overriding foo() from the base class
        // Derived implementation
    }
};
```

Overriding vs. Redefining

Overriding occurs when a derived class provides a specific implementation of a method that is already defined in its base class, and the base class method is declared as virtual. Whereas redefining occurs when a derived class provides a new implementation of a method that is defined in its base class, but the base class method is not declared as virtual.

If a method is redefined in the derived class without the virtual keyword, it hides the base class method rather than overriding it. The method to be called is determined at compile-time based on the static type of the pointer/reference.

```
class BaseClass {
public:
    void method() {
        // Base implementation
    }
};
class DerivedClass : public BaseClass {
public:
    void method() { // Redefining method from the base class
        // Derived implementation
    }
};
```

Runtime Polymorphism

Runtime polymorphism refers to the ability of a program to determine the object type dynamically during runtime and call the appropriate method implementation accordingly. The key features are:

- *Dynamic Binding*: The decision of which function to call is made at runtime based on the actual object type.
- *Flexibility*: Allows for writing generalized code that can work with objects of different derived types.

```
BaseClass* ptr = new DerivedClass();
ptr->method(); // Calls the method of DerivedClass at runtime
```

Summary

- Public inheritance facilitates the “is-a” relationship between classes.
- Overriding and virtual functions enable runtime polymorphism.
- Runtime polymorphism allows for dynamic method resolution based on object type.

Lecture review

Exception Handling

Exception handling is a mechanism to handle runtime errors such as division by zero, array out of bounds, etc. It is a way to transfer control from one part of a program to another.

In C++, exceptions are thrown using the **throw** keyword and caught using the **try-catch** block. The **try** block contains the code that may throw an exception, and the **catch** block contains the code that handles the exception.

```
try {
    // code that may throw an exception
    throw MyException();
}
catch (MyException& e) {
    // code to handle the exception
}
```

The **throw** statement can throw any type of object, including built-in types, standard library types, and user-defined types. The **catch** block can catch exceptions of a specific type or a base class type.

In the following example, the `safe_division()` function throws an exception of type `std::runtime_error` if the denominator is zero. The exception is caught in the `main` function and the error message is printed.

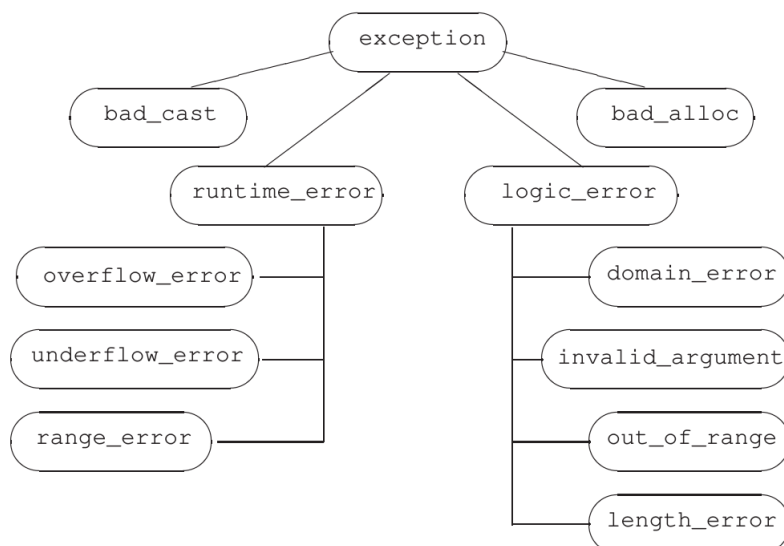
```
#include <iostream>
#include <stdexcept>
using std::cin, std::cout;

double safe_division(double a, double b) {
    if (b == 0) {
        throw std::runtime_error("Division by zero");
    }
    return a / b;
}

int main() {
    try {
        int x, y;
        cout << "Enter two numbers: ";
        cin >> x >> y;
        double result = safe_division(x, y);
        cout << "Result: " << result << "\n";
    }
    catch (std::runtime_error& e) {
        cout << "Caught an exception: " << e.what() << "\n";
    }
}
```

The `<stdexcept>` header

The `<stdexcept>` header file contains the following class hierarchy:



The base class `std::exception` has a virtual method `what()` that returns a string describing the exception. The `std::runtime_error` class is derived from `std::exception` and is used to report runtime errors. The `std::logic_error` class is used to report errors that are caused by the program logic, such as invalid arguments to a function. Further derived classes include `std::invalid_argument`, `std::domain_error`, `std::length_error`, etc.

Upcasting and Downcasting

Upcasting is the process of converting a pointer or reference of a derived class to a pointer or reference of a base class. It is done implicitly and does not require any explicit casting.

Downcasting is the process of converting a pointer or reference of a base class to a pointer or reference of a derived class. It is done explicitly using the **`static_cast`** or **`dynamic_cast`** operators.

```

class Base { /* ... */ };
class Derived : public Base { /* ... */ };

void f() {
    Derived d;
    Base* pb = &d; // upcasting
    Derived* pd = static_cast<Derived*>(pb); // downcasting
}

```

`static_cast` and `dynamic_cast`

The **`static_cast`** and **`dynamic_cast`** operators are used for type conversion between different types of pointers and references. They can be used for upcasting, downcasting, and sideways conversions between pointers or references of classes in the inheritance hierarchy. They can also be used to convert between different types of pointers, such as converting a **`void*`** pointer to another type of pointer.

The **`static_cast`** is done at compile time and does not perform any runtime type checking. While the **`dynamic_cast`** is done at runtime and performs runtime type checking to ensure that the conversion is valid. When applied to pointers, the **`dynamic_cast`** returns a null pointer if the conversion is not valid. When applied to references, it throws an exception of type `std::bad_cast` if the conversion is not valid.

```
Base* pb = new Derived;
Derived* pd = dynamic_cast<Derived*>(pb);
if (pd != nullptr) {
    // conversion is valid
}
else {
    // conversion is not valid
}

Base& rb = *pb;
try {
    Derived& rd = dynamic_cast<Derived&>(rb);
    // conversion is valid
}
catch (std::bad_cast& e) {
    // conversion is not valid
}
```

Following example uses **dynamic_cast** to do runtime type checking and downcasting to count the number of students in a vector of Person pointers.

```
class Person { /* ... */ };
class Student : public Person { /* ... */ };
class Staff : public Person { /* ... */ };

int count_students(vector<Person*>& people) {
    int count = 0;
    for (Person* p : people) {
        if (dynamic_cast<Student*>(p) != nullptr) {
            count++;
        }
    }
    return count;
}
```

Lecture review: Generic Programming (templates)

In C++, a template is a simple and yet very powerful tool. The simple idea is to pass data type as a parameter so that we don't need to write the same code for different data types.

Function templates

Let say we want to implement `abs()` function for all data types. One way is to use function overloading, but it is not a good solution because we need to write many overload functions for each data type like `int`, `float`, `double`, `long`, etc.

```
int abs(int x) {
    return (x >= 0) ? x : -x;
}
float abs(float x) {
    return (x >= 0) ? x : -x;
}
...
```

The problem with the above approach is that 1) we need to write the same code for different data types, and 2) it won't work for new user-defined data types. The generic programming provides a way to write a single *function template* that can be used by compiler to automatically generate the required function for a specific data type. A simple implementation of `abs()` function template is given below.

```
template <typename T>
T abs(T x) {
    return (x >= 0) ? x : -x;
}
```

To define a function template, the keyword **template** is used, followed by the template parameter list inside `< >` which is a comma-separated list of parameters. The keyword **typename** is used to define type parameters. The function template `abs()` can be used for any data type. For example,

```
int main() {
    int a = -5;
    float b = -5.5;
    cout << abs(a) << endl; // 5
    cout << abs(b) << endl; // 5.5
    return 0;
}
```

When calling template function, the compiler will automatically generate the required function from the template. This process is called *template instantiation*.

In the above example, since the variable `a` has type `int`, the compiler will generate a function `int abs(int x)` from the template. Similarly, the compiler will generate a function `float abs(float x)` from the template for the variable `b`.

Class templates

Like function templates, *class templates* are a useful way to define a class that can work with any data type. For example, we can define a `Vector3D` class template that can use any data type for its elements.

```
template <typename T>
class Vector3D {
    T x, y, z;
public:
    Vector3D(T x, T y, T z) : x(x), y(y), z(z) {}
    T magnitude() {
        return sqrt(x*x + y*y + z*z);
    }
};
```

The class template `Vector3D` can be used for any data type. For example,

```
int main() {
    Vector3D<int> v1(1, 2, 3);
    Vector3D<float> v2(1.5, 2.5, 3.5);
    cout << v1.magnitude() << endl; // 3.74166
    cout << v2.magnitude() << endl; // 4.30116
    return 0;
}
```

The compiler will generate two classes from the template: `Vector3D<int>` and `Vector3D<float>`.

Measure execution time of a function

To measure the execution time of a function, we can use the following code snippet.

```
#include <chrono>
using namespace std::chrono;
// note the timepoint before the function call
auto start = high_resolution_clock::now();
func(); // the function call of which we want to measure the execution time
auto stop = high_resolution_clock::now(); // time after the function call

// Subtract stop and start timepoints and cast it to required unit.
auto duration = duration_cast<microseconds>(stop - start);

cout << duration.count() << endl; // print the duration
```

In the above code, we have used **auto** keyword to avoid typing long type definitions. The `<chrono>` library provides the function `now` to get the timepoint at this instant. The `high_resolution_clock` is a clock that provides the smallest possible tick period.

The `duration_cast` function is used to convert the difference between two timepoints to a specific unit of time. Predefined units are nanoseconds, microseconds, milliseconds, seconds, minutes, and hours. To get the value of duration, we use the `count()` member function on the duration object.

Accociative Containers

map<K,V>

The map<K,V> associative container in C++ store a collection of *key-value* pairs where the keys are unique and sorted in ascending order. It is generic in the sense that the type of the key K and the type of the value V can be any type.

The map<K,V> container has the following member functions (and more):

Function	Description	Usage example
insert()	Inserts a pair of the form {key,value} into the map. The key must be unique. Ignore if key is already present in the map.	erp.insert({"Ali",12345})
operator[]	Returns a reference to the value associated with the given key. If the key is not present, it inserts a pair {key,x} where x is default value of type V and returns a reference to the value x.	cout << erp["Ali"] erp["Ali"]=12543
find()	Returns an iterator to the element with the given key. If key is not present in the map, it returns end()	if(erp.find("Zia")==erp.end()) cout << "Not found"
contains()	Returns true if the map contains the given key (C++20)	if(!erp.contains("Zia")) cout << "Not found"
erase()	Removes the element with the given key	erp.erase("Ali")
size()	Returns the number of elements in the map	erp.size()
empty()	Returns true if the map is empty	erp.empty()
clear()	Removes all elements from the map	erp.clear()

The STL library contains an unordered variant of the map<K,V> container called unordered_map<K,V>. The unordered_map<K,V> container is defined in the <unordered_map> header file. It has the all the member functions described above.

In the following example, we use map<string,int> to store erp id of different students. The keys are the names of the students and the values are their erp ids. The keys are sorted in ascending order. The map<K,V> container is defined in the <map> header file.

```
#include <iostream>
#include <string>
#include <map>
using std::cout, std::endl, std::map, std::string;

int main() {
    map<string, int> erp;
    erp["Ali"] = 12345;    // insert pair ("Ali", 12345)
    erp["Ali"] = 13901;    // update value of key "Ali" to 13901
    erp["Beena"] = 12897; // insert pair ("Beena", 12897)
    erp.insert({"Beena", 14593}); // ignore, key "Beena" is already present
```

```

erp.insert({"Chand", 13412}); // insert pair ("Chand", 13412)

cout << erp["Ali"] << endl;    // prints 13901
cout << erp["Beena"] << endl;  // prints 12897
cout << erp["Chand"] << endl;  // prints 13412
if(erp.find("Dua")==erp.end())
    cout << "Not present\n";
cout << erp["Dua"] << endl;    // "Dua" is not present, insert pair ("Dua", 0), prints 0

// Iterate using C++17
for (auto& [key, value] : erp)
    std::cout << '[' << key << "] = " << value << "; ";

// C++11 alternative:
// for (auto& n : erp)
//     std::cout << n.first << " = " << n.second << "; ";
}

```

set<K>

The `set<K>` associative container in C++ stores a collection of unique keys that are sorted in ascending order. It is generic in the sense that the type of the key `K` can be any type.

The `set<K>` container has the following member functions (and more):

Function	Description	Usage example
<code>insert()</code>	Inserts a key into the set. The key must be unique. Ignore if key is already present in the set.	<code>itp.insert("Ali")</code>
<code>find()</code>	Returns an iterator to the element with the given key. If key is not present in the set, it returns <code>end()</code>	<code>if(itp.find("Zia")==itp.end()) cout << "Not found"</code>
<code>contains()</code>	Returns true if the set contains the given key (C++20)	<code>if(!itp.contains("Zia")) cout << "Not found"</code>
<code>erase()</code>	Removes the element with the given key	<code>itp.erase("Ali")</code>
<code>size()</code>	Returns the number of elements in the set	<code>itp.size()</code>
<code>empty()</code>	Returns true if the set is empty	<code>itp.empty()</code>
<code>clear()</code>	Removes all elements from the set	<code>itp.clear()</code>

The STL library contains an unordered variant of the `set<K>` container called `unordered_set<K>`. The `unordered_set<K>` container is defined in the `<unordered_set>` header file. It has all the member functions which are described above.

In the following example, we use `set<string>` to store names of different students. The keys are the names of the students. The keys are sorted in ascending order. The `set<K>` container is defined in the `<set>` header file.

```

#include <iostream>
#include <string>

```

```

#include <set>
using std::cout, std::endl, std::set, std::string;

int main() {
    set<string> itp;
    itp.insert("Ali"); // insert key "Ali"
    itp.insert("Ali"); // ignore, key "Ali" is already present
    itp.insert("Beena"); // insert key "Beena"
    itp.insert("Chand"); // insert key "Chand"

    cout << itp.size() << endl; // prints 3
    if(itp.find("Dua")==itp.end())
        cout << "Not present\n";
    itp.erase("Ali"); // remove key "Ali"
    cout << itp.size() << endl; // prints 2

    return 0;
}

```

Reading from text file

The following program reads a text file line by line and prints the lines on the standard output. The `ifstream` class is defined in the `<fstream>` header file.

```

#include <iostream>
#include <fstream>
#include <string>
using std::cout, std::endl, std::ifstream, std::string;

int main() {
    ifstream ifs("input.txt"); // assuming input.txt is in the current directory
    if(!ifs.is_open()) {
        cout << "Error opening file\n";
        return 1;
    }
    // ifs can be used like cin
    // e.g. int x; ifs >> x; // read an integer from file
    string line;
    while(getline(ifs, line)) {
        cout << line << endl;
    }
    return 0;
}

```

Generating random numbers

The `<random>` library in C++ provides a number of random number generators and distributions. The random number generators are used to generate random numbers (usually 32 or 64 bit unsigned **int** values), while the distributions are used to convert these numbers to a specific distribution, i.e, uniform, normal, etc. A good tutorial on random number generation in C++ is available at: <https://hackingcpp.com/cpp/std/random.html>. See also https://hackingcpp.com/cpp/std/random_distributions.png for a cheat sheet summary of random number distributions available in `<random>` library.

```
#include <iostream>
#include <random>

int main() {
    // ***** generator *****
    // generate a random seed (possibly slow on some systems)
    auto const seed = std::random_device{}(); // constant int value could be used
        for reproducing the same random numbers in every run of the program

    // declare and seed the psedu-random generator
    auto generator = std::default_random_engine {seed};

    // ***** distributions *****
    // generate uniform int: [1, 100] inclusive
    auto uniform_int = std::uniform_int_distribution<int> {1, 100};

    // generate random floats in [-1.2, 6.25), lower bound inclusive, upper bound
        exclusive
    std::uniform_real_distribution<float> uniform_float {-1.2f, 6.25f};

    // true/false with 60% chance of true
    auto flip = std::bernoulli_distribution {0.6};

    // normal distribution with mean 0.5 and standard deviation 2
    auto norm = std::normal_distribution<double>{0.5, 2};

    // ***** generate random numbers *****
    // generate 10 uniform random ints between 1 and 100
    for(int i = 0; i < 10; ++i)
        std::cout << uniform_int(generator) << " ";
    std::cout << "\n-----\n";

    // generate 10 uniform random floats between -1.2f to 6.25f
```



```
for(int i = 0; i < 10; ++i)
    std::cout << uniform_float(generator) << " ";
std::cout << "\n-----\n";

// generate 10 coin tosses (probability of head is 60%)
for(int i = 0; i < 10; ++i)
    std::cout << ( flip(generator) ? "Head" : "Tail") << " ";
std::cout << "\n-----\n";

// generate 10 normal random numbers with mean 0.5 and standard deviation 2
for(int i = 0; i < 10; ++i)
    std::cout << norm(generator) << " ";
std::cout << "\n-----\n";
}
```

Lecture Review

See the following links for a review of STL algorithms: <https://hackingcpp.com/cpp/std/algorithms.html>

For range-based algorithms, see: https://hackingcpp.com/cpp/std/range_algorithms.html

Or, for more beginner friendly guide, see: https://hackingcpp.com/cpp/beginners_guide.html